

Ansible Installation

Open Powershell as Administrator and give the command

wsl --install

This command used to download **WSL -(Windows supporting for Linux)** this console used to develop or work with ansible .

Go to Microsoft Store Install **Ubuntu**

After complete the installation open wsl (go to start > search wsl & open)

To update the packages ,

Sudo apt update && sudo apt upgrade

To install ansible ,

Sudo apt update && sudo apt install ansible -y

(optional) If above command show error like this “ **waiting for cache lock : could not get lock /var/lib/dpkg/lock frontend It is held process 641**”

So you need to kill locking process ,

Ps aux | grep apt – To identify process

Kill process ,

Sudo Kill -9 641

Remove lock files ,

Sudo rm /var/lib/apt/lists/lock

Sudo rm /var/lib/dpkg/

Sudo rm /var/lib/dpkg/lock - frontend

After that again install , now you can install ansible without error,

Sudo apt update && sudo apt install ansible -y

IAM user creation

- In left side of the you can see IAM users click it .
- Then create user & click Next .
- In search bar type ec2 and scroll down .
- **Enable** AmazonEc2 Full Access
- Now scroll down bottom of the page you can see click Next to create user

Now you had created user with name as you given , so click that user

- Go to security credentials
- Click create accesskey
- **Enable** Application running outside Aws
- Then Click Next > Create Access key
- Now you got **Access key** , **secret key** copy those keys and save it on notepad for later use .

Open wsl ,

Mkdir Ans1 – create folder where you want , then open vs code using **code** .

- After that open terminal on vs code
- Check python is installed **python --version** if not install it

Give the below commands ,

- **Sudo apt install python3-pip**

- **ansible-galaxy collection install amazon-aws** – sometimes it may have already installed .
- **openssl rand –base64 2048 > vault.pass** - it is used to create vault file used to protect sensible content like a access key & secret key .
- **Ansible-vault create group_vars/all/pass.yml --vault --password –file vault.pass** -to create pass.yml file it will open the file inside the thr terminal like a editor after that type your keys like ,

ec2_access_key : AKIATX3PHS3RLXTLUYV3

ec2_secret_key : uqP13Cj6/EHtIciz9ZFMPGD5EdLkNVPheLWgDHj8

After that click **ESC** and **:wq** for write and execute .

- **Ansible-vault edit group_vars/all/pass.yml --vault --password –file vault.pass** - this command used to open the pass file and edit it .

Now you want to create yaml file within the folder here named as **ec2_create.yaml** . Inside the file give (code)instruction to create aws instance

```
---
- hosts: localhost
  vars_files:
    - group_vars/all/pass.yml
  connection: local
  gather_facts: yes

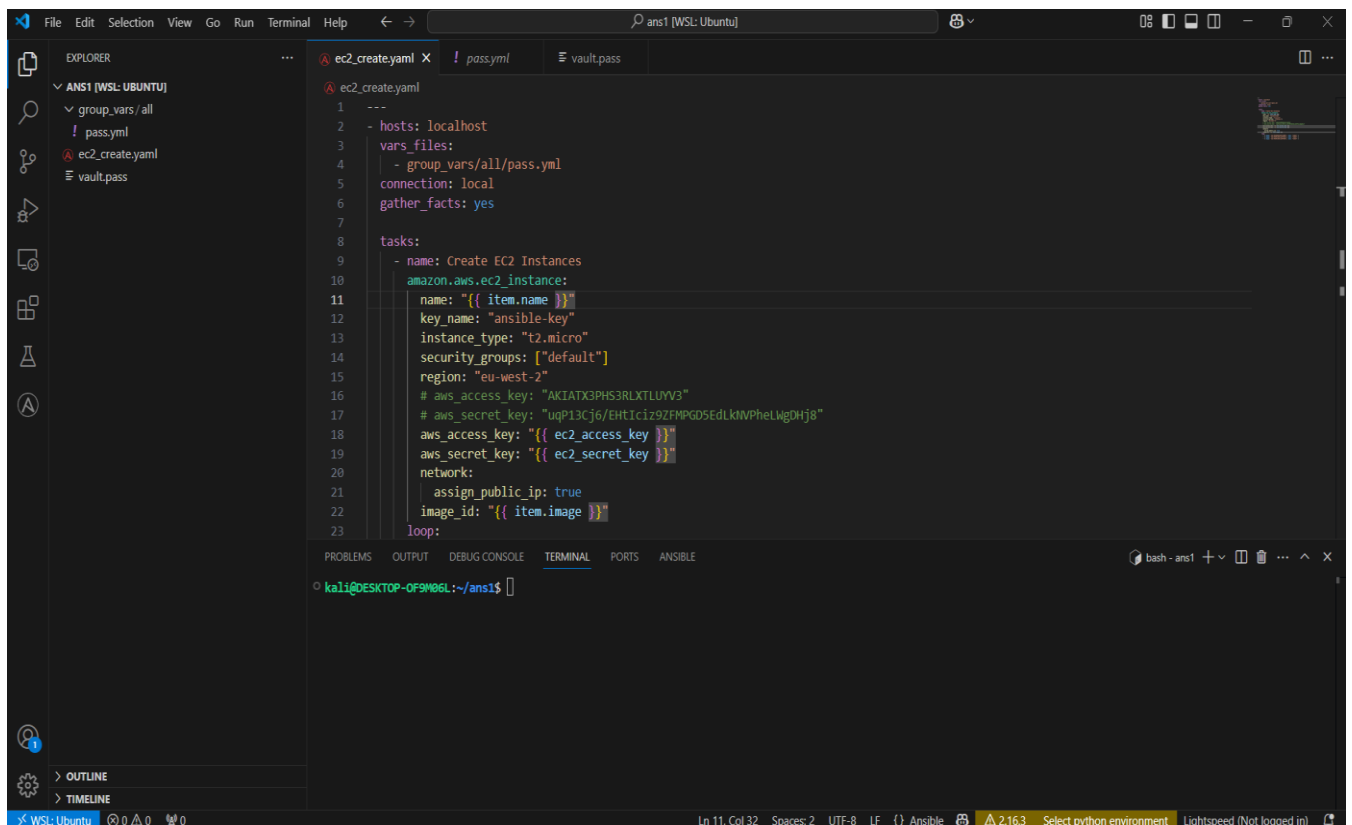
  tasks:
    - name: Create EC2 Instances
      amazon.aws.ec2_instance:
        name: "{{ item.name }}"
        key_name: "ansible-key"
        instance_type: "t2.micro"
        security_groups: ["default"]
        region: "eu-west-2"
        # aws_access_key: "AKIATX3PHS3RLXTLUYV3"
        # aws_secret_key: "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"
        aws_access_key: "{{ ec2_access_key }}"
        aws_secret_key: "{{ ec2_secret_key }}"
        network:
          assign_public_ip: true
        image_id: "{{ item.image }}"
```

```

loop:
  - { image: "ami-0e56583ebfdcf098f", name: "node1" }
  - { image: "ami-0a94c8e4ca2674d5a", name: "node2" }
  - { image: "ami-0a94c8e4ca2674d5a", name: "node3" }

```

Below given screenshot is show the file structure ,



After write the code within the terminal give command for execute the playbook file . the command is ,

ansible-playbook ec2_create.yml --vault-password --file vault.pass

This command was execute playbook file here after execution 3 instance was created on your AWS account .

```
File Edit Selection View Go Run Terminal Help →
[?] ans1 [WSL: Ubuntu]
EXPLORER
ANS1 [WSL: UBUNTU]
  group_vars/all
  ! pass.yml
  ec2_create.yaml
  vault.pass
ec2_create.yaml
2 - hosts: localhost
10 amazon.aws.ec2_instance:
11   name: "{{ item.name }}"
12   key_name: "ansible-key"
13   instance_type: "t2.micro"
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS ANSIBLE
kali@DESKTOP-OF9M06L:~/ans1$ ansible-playbook ec2_create.yaml --vault-password-file vault.pass
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that the implicit localhost does not match 'all'

PLAY [localhost] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Create EC2 Instances] *****
changed: [localhost] => (item={'image': 'ami-0e56583ebfdcf098f', 'name': 'managed-node1'})
changed: [localhost] => (item={'image': 'ami-0a94c8e4ca2674d5a', 'name': 'managed-node2'})
changed: [localhost] => (item={'image': 'ami-0a94c8e4ca2674d5a', 'name': 'managed-node3'})

PLAY RECAP *****
localhost          : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

kali@DESKTOP-OF9M06L:~/ans1$
```

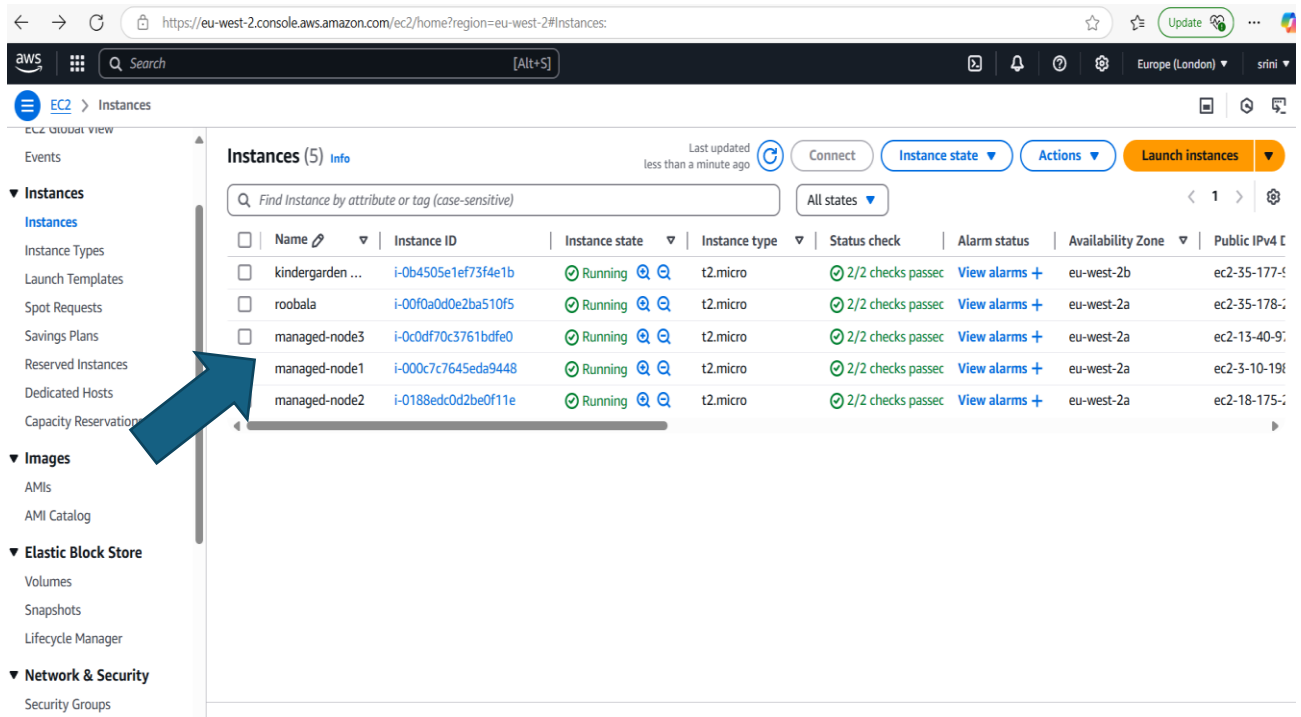
Note :

If suppose you need to create 3 instance with same image id ,better way is create tags below the image id .

tags:

environment : “{{ item.name }}

Because some time ansible does not create same image id again and again . So in that environment variable seperate the each instance with their environment variable .



Stop Instance Using Ansible playbook

Open wsl ,

Ensure your within the folder . Here i am in ans1 folder which i used to create instance .

ls- Using this command check all files in that folder and Make sure that have pem file . If no pem file you should be copy here,

```
Mkdir -p ~/ans1
```

This command used to create directory , -p is parent flag check parant node have this path .

```
Cp/mnt/c/users/Nextwebby\Mind\power/downloads/ansible-key.pem  
~/ans1/
```

This command used to copy the pem file from windows to wsl .

(If you doesn't have pem file download ppk file (keypair) from aws and generate pem file via **Puttygen** .)

Now check using **ls** command you can see pem file in your folder .

Chmod 600 ~/ans1/ansible-key.pem

This command for change permission . 6 – file owner can read and write , 0- No permission for group , 0 – No permission for others .

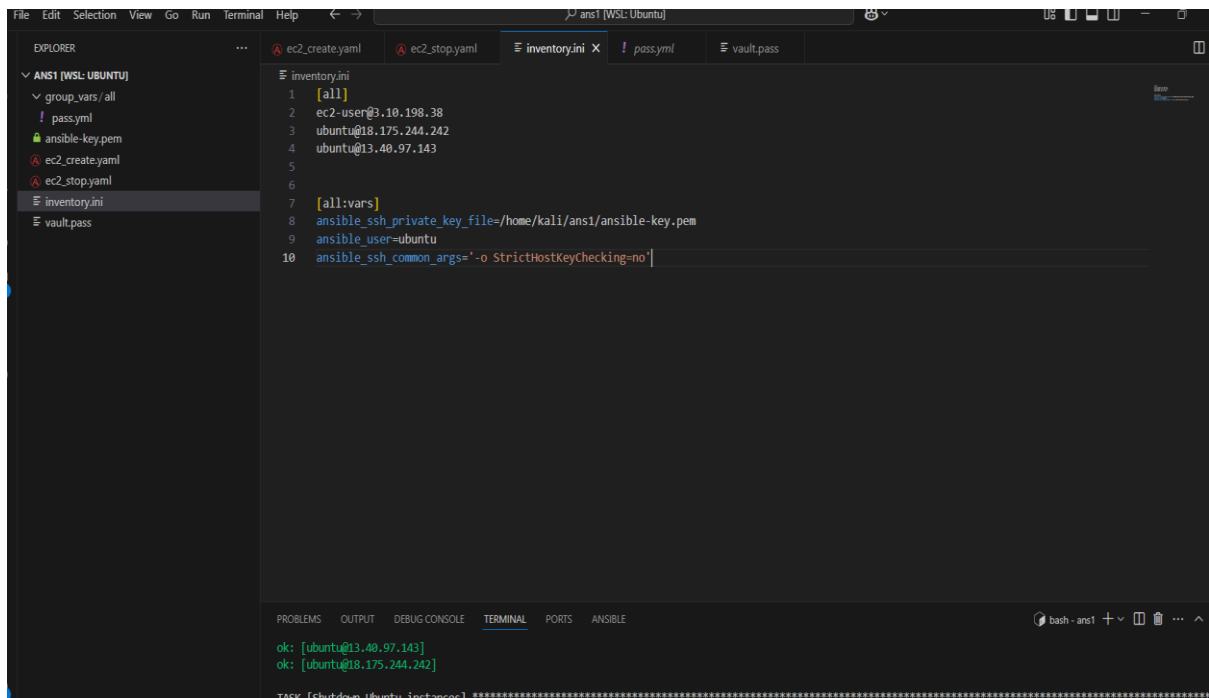
Now Open **vs code** ,

Open ans1 folder within that create **inventory.ini** file. This file used to store manged node , group details .below is code for ini file,

```
[all]
ec2-user@3.10.198.38
ubuntu@18.175.244.242
ubuntu@13.40.97.143

[all:vars]
ansible_ssh_private_key_file=/home/kali/ans1/ansible-key.pem
ansible_user=ubuntu
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

Ansible_ssh_common_args this variable used to configure , -o is ssh option disable the ssh hostkey verification .



Create yaml file for stop instance , here(**ec2_stop.yml**)

```
---
- hosts: all
  become: true
  gather_facts: yes
  ignore_unreachable: yes
  ignore_errors: yes

  tasks:
    - name: Shutdown Ubuntu instances
      ansible.builtin.command: /sbin/shutdown -h now
      async: 1
      poll: 0
      when: ansible_facts['os_family'] == "Debian"
      ignore_errors: yes
      register: shutdown_result

    - name: End session for shutdown hosts
      ansible.builtin.meta: end_host
      when: shutdown_result is defined
```

Explanation for some keywords ,

become:true it used to run with sudo

-h now – for immediatly shutdown

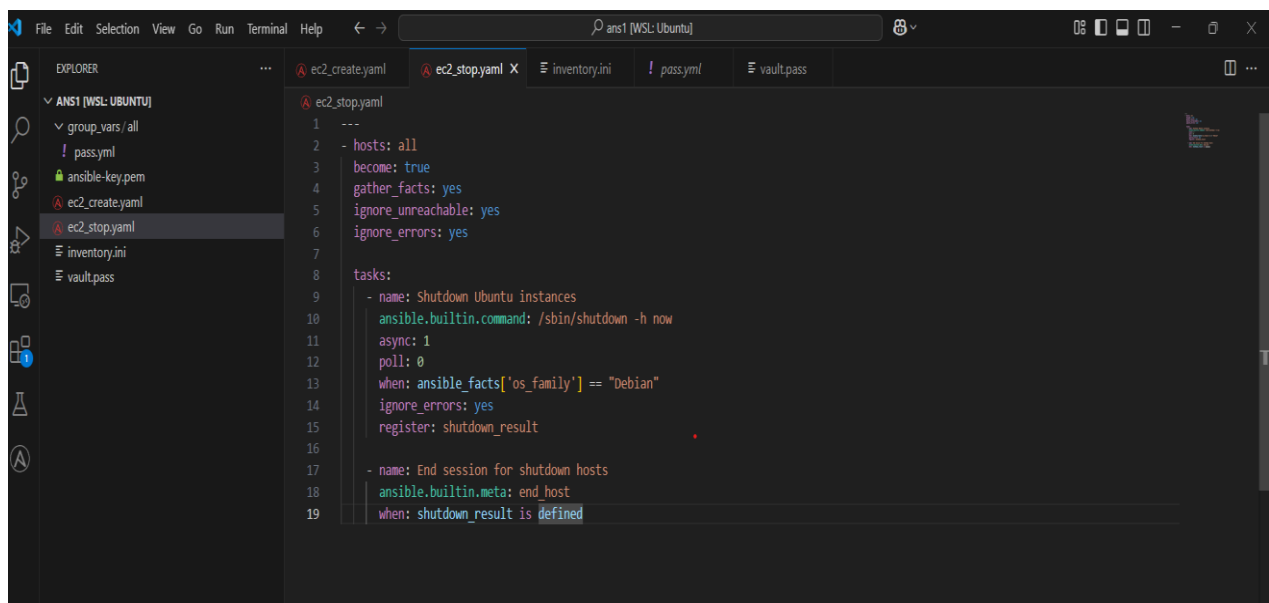
async:1 used to asynchronously continue without waiting .

Poll : 0 tell not to wait

register :shutdown_result store the task result on shutdown_result variable

Ansible.builtin.meta: end_host here ansible stop running further task on that host

When : shutdown_result is defined it like a if condition . This when condition ensure if shutdown task successfully initiated the stop all process .

A screenshot of the Visual Studio Code editor interface. The Explorer sidebar on the left shows a file tree for 'ANS1 [WSL: UBUNTU]' with files like 'group_vars/all', 'pass.yml', 'ansible-key.pem', 'ec2_create.yml', 'ec2_stop.yml' (selected), 'inventory.ini', and 'vault.pass'. The main editor window displays the content of 'ec2_stop.yml'. The file content is as follows:

```
1 ---
2 - hosts: all
3   become: true
4   gather_facts: yes
5   ignore_unreachable: yes
6   ignore_errors: yes
7
8   tasks:
9     - name: Shutdown Ubuntu instances
10       ansible.builtin.command: /sbin/shutdown -h now
11       async: 1
12       poll: 0
13       when: ansible_facts['os_family'] == "Debian"
14       ignore_errors: yes
15       register: shutdown_result
16
17     - name: End session for shutdown hosts
18       ansible.builtin.meta: end_host
19       when: shutdown_result is defined
```

After that run the ec2_stop.yml file ,

Ansible-playbook -i inventory.ini ec2_stop.yml --vault --password-file vault.pass

```

kali@DESKTOP-OF9M6EL:~/ansible$ ansible-playbook -i inventory.ini ec2_stop.yml --vault-password-file vault.pass
PLAY [all]

TASK [Gathering Facts] *****
fatal: [ec2-user@3.10.198.38]: UNREACHABLE! => ("changed": false, "msg": "Failed to connect to the host via ssh: ubuntu@3.10.198.38: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).", "unreachable": true)
...ignoring
ok: [ubuntu@13.40.97.143]
ok: [ubuntu@18.175.244.242]

TASK [Shutdown Ubuntu instances] *****
fatal: [ec2-user@3.10.198.38]: FAILED! => ("msg": "The conditional check 'ansible facts[os family] == \Debian'' failed. The error was: error while evaluating conditional (ansible facts[os family] == \Debian\"): 'dict object' has no attribute 'os family'. 'dict object' has no attribute 'os family'\n\nthe error appears to be in '/home/kali/ansible/ec2_stop.yml': line 9, column 7, but may/nbe elsewhere in the file depending on the exact syntax problem.\n\nthe offending line appears to be:\n\n  tasks:\n    - name: Shutdown Ubuntu instances\n      ^ here\n")
...ignoring
changed: [ubuntu@13.40.97.143]
changed: [ubuntu@18.175.244.242]

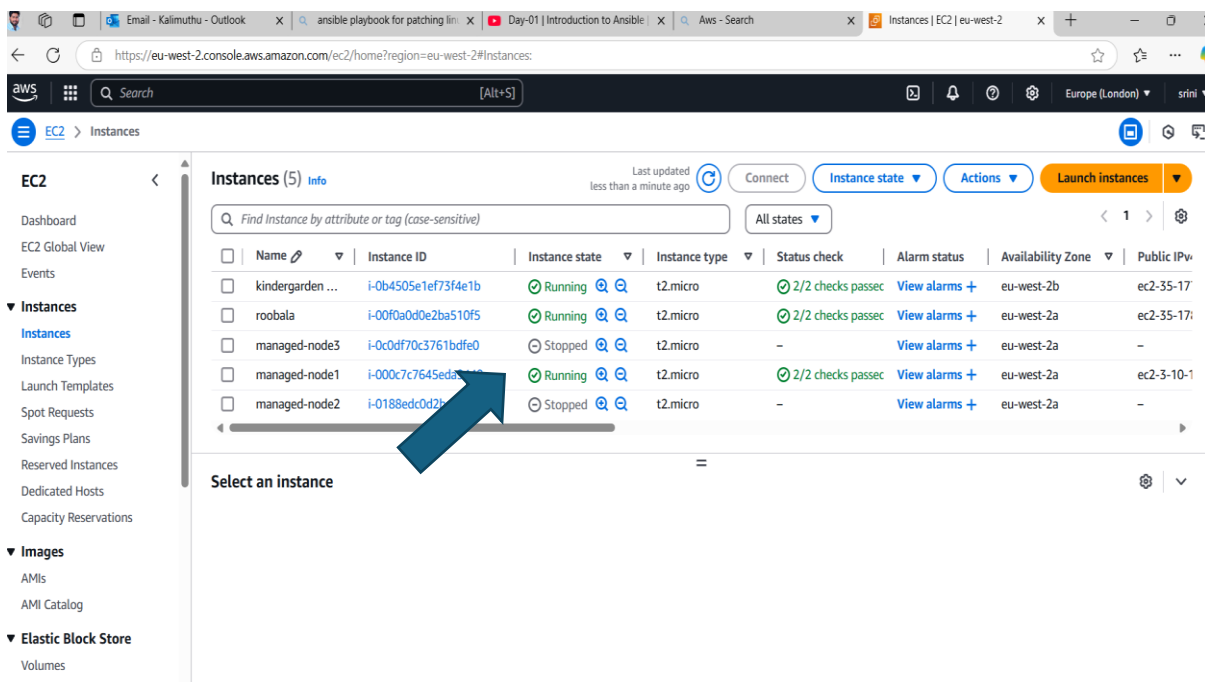
TASK [End session for shutdown hosts] *****

TASK [End session for shutdown hosts] *****

TASK [End session for shutdown hosts] *****

PLAY RECAP *****
ec2-user@3.10.198.38 : ok=2 changed=0 unreachable=0 failed=0 skipped=0 rescued=0 ignored=2
ubuntu@13.40.97.143 : ok=2 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
ubuntu@18.175.244.242 : ok=2 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0

```



Now , **Manage-node 2 & Manage-node 3** was stopped . Because it was stopped based on their os family .

Both instance are **Debian** based os family and **Managed-node 1** is **Aws linux** this is **RedHat** os Family.

Start instance via Playbook

Create new yaml file to start instance here it maned as **ec2_start .yaml**

Then write code for start the aws instance

```
---
- hosts: localhost
  vars_files:
    - group_vars/all/pass.yml
  connection: local
  gather_facts: no

  tasks:
    - name: Start stopped Ubuntu EC2 instances
      amazon.aws.ec2_instance:
        instance_ids:
          - "i-0188edc0d2be0f11e"
          - "i-0c0df70c3761bdfe0"
        state: running
        region: "eu-west-2"
        aws_access_key: "{{ ec2_access_key }}"
        aws_secret_key: "{{ ec2_secret_key }}"
        register: ec2_start
        ignore_errors: yes

    - name: Wait for instances to be running
      amazon.aws.ec2_instance_info:
        instance_ids:
          - "i-0188edc0d2be0f11e"
          - "i-0c0df70c3761bdfe0"
        region: "eu-west-2"
        register: instance_info
        until: instance_info.instances | selectattr('state', 'equalto',
'running') | list | length == instance_info.instances | length
        retries: 30
        delay: 10
        ignore_errors: yes
        when: ec2_start is changed
```

Here **–name : wait for instance to be running** this task is optional . It is used to hold the task until it changes to running state .

until: instance_info.instances | selectattr('state', 'equalto', 'running') | list | length == instance_info.instances | length this line is used to keep check until all instance running .

Retries :30 , delay :10 – it will check 30 times with 10s delay .

After that run the file

Ansible-playbook -i inventory.ini ec2_start.yaml --vault --password-file vault.pass

```
ec2_start.yaml
2  - hosts: localhost
9  tasks:
10 - name: Start stopped Ubuntu EC2 instances
20   ignore_errors: yes
21
22 - name: Wait for instances to be running
23   amazon.aws.ec2_instance_info:
24     instance_ids:
25       - "i-0188edc0d2be0f11e"
26       - "i-0c0df70c3761bdf00"
27     region: "eu-west-2"
28     register: instance_info
29     until: instance_info.instances | selectattr('state', 'equalto', 'running') | list | length == instance_info.instances | length
30     retries: 30

PLAY RECAP *****
localhost                : ok=1  changed=0  unreachable=0  failed=0  skipped=1  rescued=0  ignored=1

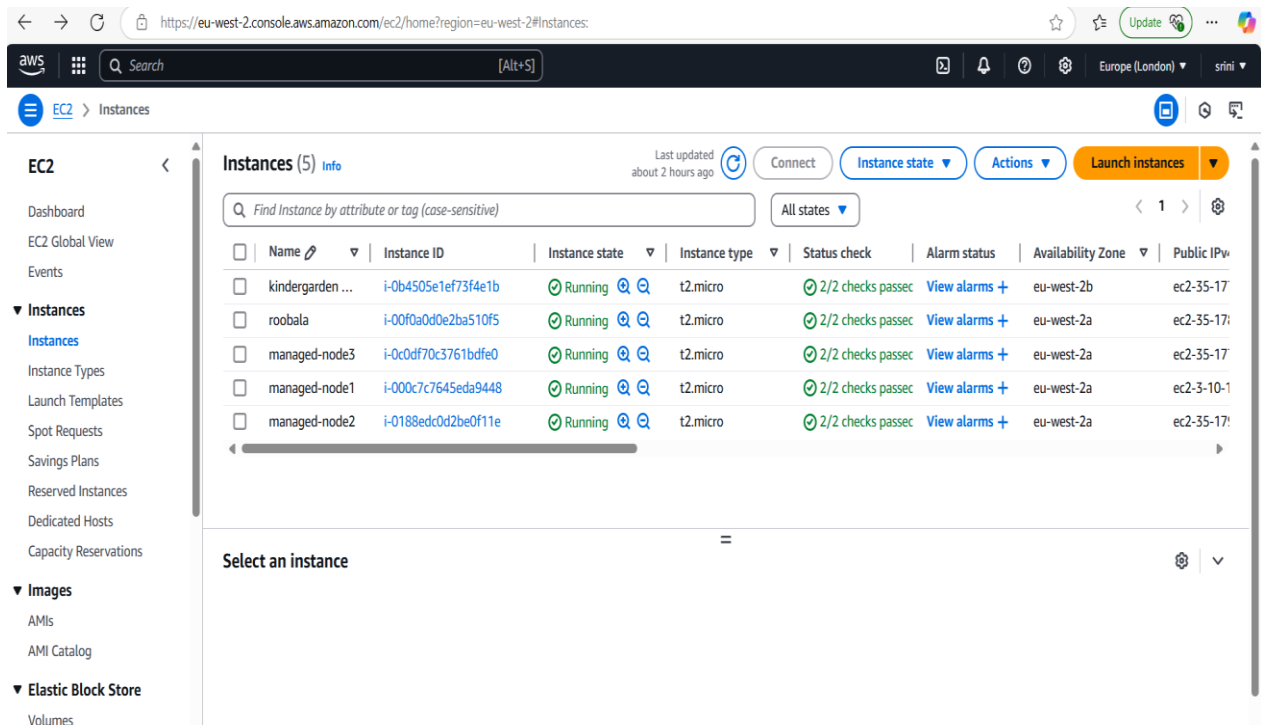
❯ kali@DESKTOP-OF9M06L:~/ans1$ ansible-playbook -i inventory.ini ec2_start.yaml --vault-passwd-file vault.pass

PLAY [localhost] *****

TASK [Start stopped Ubuntu EC2 instances] *****
changed: [localhost]

TASK [Wait for instances to be running] *****
fatal: [localhost]: FAILED! => {"msg": "The conditional check 'instance_info.instances | selectattr('state', 'equalto', 'running') | list | length == instance_info.instances | length' failed. The error was: error while evaluating conditional (instance_info.instances | selectattr('state', 'equalto', 'running') | list | length == instance_info.instances | length): 'dict object' has no attribute 'instances'. 'dict object' has no attribute 'instances'" }
...ignoring

PLAY RECAP *****
localhost                : ok=2  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=1
```



Install httpd on Aws linux OS

Open the inventory file ensure that have **aws:linux** group . If not give the with public IP like below ,

```
[all]
ec2-user@3.10.198.38 ansible_user=ec2-user
ubuntu@18.171.144.25
ubuntu@18.134.253.246

[all:vars]
ansible_ssh_private_key_file=/home/kali/ans1/ansible-key.pem
ansible_user=ubuntu
ansible_ssh_common_args='-o StrictHostKeyChecking=no -o ConnectTimeout=30'
```



```
[aws_linux]
aws_linux_instance ansible_host=3.10.198.38 ansible_user=ec2-user
ansible_ssh_private_key_file=/home/kali/ans1/ansible-key.pem
```

Then create yaml file for perform the tasks here it was named as **install_httpd.yaml**

```
- name: Install and configure HTTPD on AWS Linux
  hosts: aws_linux
  gather_facts: no
  become: yes

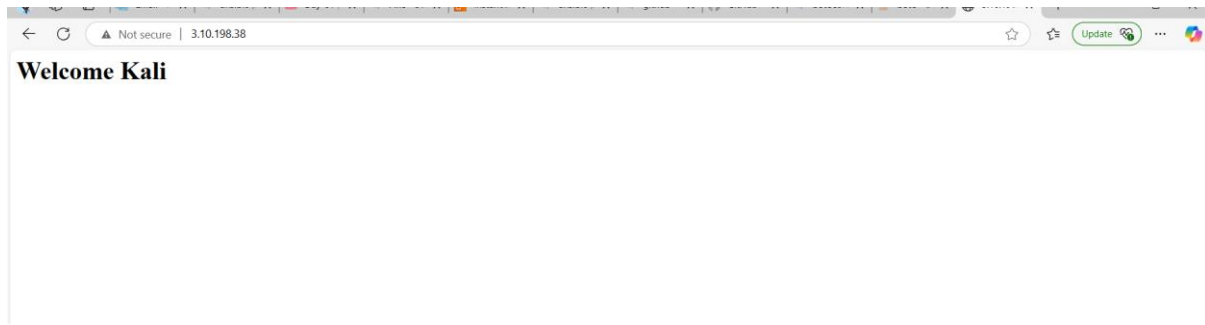
  tasks:
    - name: Install httpd
      yum:
        name: httpd
        state: present
        update_cache: yes

    - name: Start and enable httpd service
      service:
        name: httpd
        state: started
        enabled: yes

    - name: Verify httpd is running
      uri:
        url: "http://localhost"
        return_content: yes
      register: httpd_result
      until: httpd_result.status == 200
      retries: 5
      delay: 3
```

Now run the file ,

Ansible-playbook -i inventory.ini install_httpd.yaml --vault --password-file vault.pass



You can also check via **WSL** console ,

Ssh -i ~/.ssh/ansible-key.pem ec2-user@3.10.198.38

This command used to connect you aws instance in wsl . Then ,

Sudo yum list installed httpd

This command used to check httpd is already installed or not

PIP and Django installation

Create another one yaml file (playbook) for install pip & django

```
---
- name: Install Django & pip
  hosts: aws_linux
  gather_facts: no
  become: yes

  tasks:
    - name: Install pip
      yum:
        name: pip
        state: present
        update_cache: yes

    - name: Install Django
      pip:
        name: django
        state: present
        executable: pip3
```

executable : pip3 this is used to install specific version of pip package .

Now run the file

ansible-playbook -i inventory.ini install_django.yaml --vault-password-file vault.pass

```
File Edit Selection View Go Run Terminal Help  ← →  ansi [WSL: Ubuntu]
EXPLORER  ...  ! ec2_create.yaml  ! install_httpd.yaml  ! install_django.yaml X  inventory.ini  ! pass.yaml  vault.pass
ANSI [WSL: UBUNTU]
  group_vars/all
  ! pass.yaml
  ! ansible-key.pem
  ! ec2_create.yaml
  ! ec2_start.yaml
  ! ec2_stop.yaml
  ! install_django.yaml
  ! install_httpd.yaml
  inventory.ini
  vault.pass

install_django.yaml
1 ---
2 - name: Install Django & pip
3   hosts: aws_linux
4   gather_facts: no
5   become: yes

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ANSIBLE
kali@DESKTOP-OF9M06L:~/ansi$ sudo yum list installed httpd
[sudo] password for kali:
sudo: yum: command not found
kali@DESKTOP-OF9M06L:~/ansi$ ansible-playbook -i inventory.ini install_django.yaml --vault-password-file vault.pass

PLAY [Install Django & pip] *****

TASK [Install pip] *****
fatal: [ansible_ssh_private_key_file=/home/kali/ansi/ansible-key.pem]: UNREACHABLE! => {"changed": false, "msg": "Failed to connect to the host via ssh: ssh: Could not resolve hostname ansible_ssh_private_key_file=/home/kali/ansi/ansible-key.pem: Name or service not known", "unreachable": true}
[WARNING]: Platform linux on host aws_linux_instance is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for more information.
changed: [aws_linux_instance]

TASK [Install Django] *****
fatal: [aws_linux_instance]: FAILED! => {"changed": false, "msg": "Unsupported parameters for (pip) module: update_cache. Supported parameters include: chdir, executable, extra_args, name, requirements, state, umask, version, virtualenv, virtualenv_command, virtualenv_python, virtualenv_site_packages."}

PLAY RECAP *****
ansible_ssh_private_key_file=/home/kali/ansi/ansible-key.pem : ok=0  changed=0  unreachable=1  failed=0  skipped=0  rescued=0  ignored=0
aws_linux_instance      : ok=1  changed=1  unreachable=0  failed=1  skipped=0  rescued=0  ignored=0
```

```
more information.
fatal: [aws_linux_instance]: FAILED! => {"ansible_facts": {"discovered_interpreter_python": "/usr/bin/python3.9"}, "changed": false, "failures": ["No package django available."], "msg": "Failed to install some of the specified packages", "rc": 1, "results": []}

PLAY RECAP *****
ansible_ssh_private_key_file=/home/kali/ansi/ansible-key.pem : ok=0  changed=0  unreachable=1  failed=0  skipped=0  rescued=0  ignored=0
aws_linux_instance      : ok=0  changed=0  unreachable=0  failed=1  skipped=0  rescued=0  ignored=0

kali@DESKTOP-OF9M06L:~/ansi$ ansible-playbook -i inventory.ini install_django.yaml --vault-password-file vault.pass

PLAY [Install Django & pip] *****

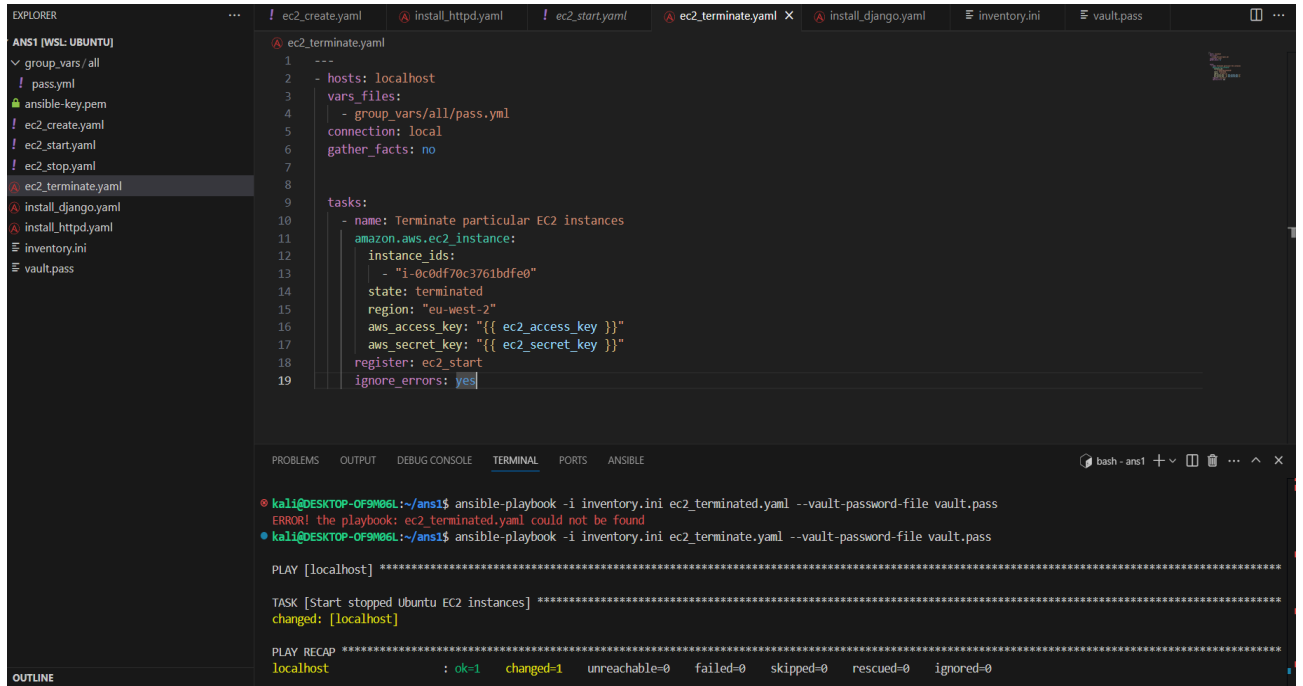
TASK [Install Django] *****
fatal: [ansible_ssh_private_key_file=/home/kali/ansi/ansible-key.pem]: UNREACHABLE! => {"changed": false, "msg": "Failed to connect to the host via ssh: ssh: Could not resolve hostname ansible_ssh_private_key_file=/home/kali/ansi/ansible-key.pem: Name or service not known", "unreachable": true}
[WARNING]: Platform linux on host aws_linux_instance is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for more information.
changed: [aws_linux_instance]

PLAY RECAP *****
ansible_ssh_private_key_file=/home/kali/ansi/ansible-key.pem : ok=0  changed=0  unreachable=1  failed=0  skipped=0  rescued=0  ignored=0
aws_linux_instance      : ok=1  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

kali@DESKTOP-OF9M06L:~/ansi$
```

Now you can check the instance django should be installed

ansible-playbook -i inventory.ini ec2_terminate.yaml --vault-password-file vault.pass



```
1 ---
2 - hosts: localhost
3   vars_files:
4     - group_vars/all/pass.yml
5   connection: local
6   gather_facts: no
7
8
9 tasks:
10  - name: Terminate particular EC2 instances
11    amazon.aws.ec2_instance:
12      instance_ids:
13        - "i-0c0df70c3761bdf0"
14      state: terminated
15      region: "eu-west-2"
16      aws_access_key: "{{ ec2_access_key }}"
17      aws_secret_key: "{{ ec2_secret_key }}"
18      register: ec2_start
19      ignore_errors: yes
```

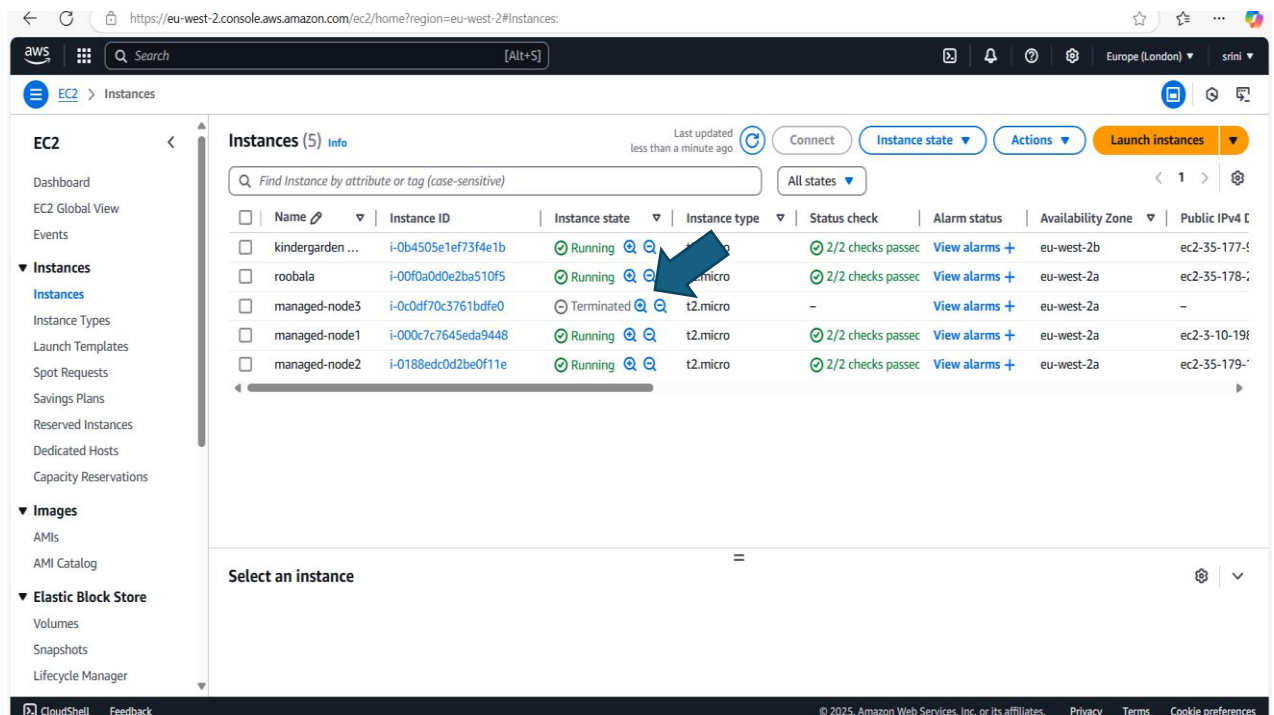
PLAY [localhost] *****

TASK [Start stopped Ubuntu EC2 instances] *****

changed: [localhost]

PLAY RECAP *****

localhost : ok=1 changed=1 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0



Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
kindergarden ...	i-0b4505e1ef73f4e1b	Running	t2.micro	2/2 checks passed	View alarms +	eu-west-2b	ec2-35-177-...
roobala	i-00f0a0d0e2ba510f5	Running	t2.micro	2/2 checks passed	View alarms +	eu-west-2a	ec2-35-178-...
managed-node3	i-0c0df70c3761bdf0	Terminated	t2.micro	-	View alarms +	eu-west-2a	-
managed-node1	i-000c7c7645eda9448	Running	t2.micro	2/2 checks passed	View alarms +	eu-west-2a	ec2-3-10-19f...
managed-node2	i-0188edc0d2be0f11e	Running	t2.micro	2/2 checks passed	View alarms +	eu-west-2a	ec2-35-179-...

Here the manage-node3 (ubuntu) is terminated .

Create universal playbook file

Previously we have create seperate playbook for each operations .Now we going to be create one yaml file within that give all tasks in same file which means **create instance , install httpd , start httpd , install pip , and istall python .** (Here the file named as universal.yaml)

```
---
- hosts: localhost
  vars_files:
    - group_vars/all/pass.yml
  connection: local
  gather_facts: no

  tasks:
    - name: Create EC2 Instances
      amazon.aws.ec2_instance:
        name: "KGN"
        key_name: "ansible-key"
        instance_type: "t2.micro"
        security_groups: ["default"]
        region: "eu-west-2"
        aws_access_key: "{{ ec2_access_key }}"
        aws_secret_key: "{{ ec2_secret_key }}"
        network:
          assign_public_ip: true
        image_id: "ami-0e56583ebfdafc098f"
        wait: yes
        wait_timeout: 600
      register: ec2

    - name: Debug instance information
      debug:
        var: ec2

    - name: Wait for SSH to be available
      wait_for:
        host: "{{ ec2.instances[0].public_ip_address }}"
        port: 22
        delay: 10
        timeout: 300
      connection: local

    - name: Add new instance to host group
      add_host:
        hostname: "{{ ec2.instances[0].public_ip_address }}"
```

```
    groups: aws_instances
    ansible_user: ec2-user
    ansible_ssh_private_key_file: "~/ans1/ansible-key.pem"
    ansible_ssh_common_args:"-o StrictHostKeyChecking=no -o
UserKnownHostsFile=/dev/null"

- hosts: aws_instances
  become: yes
  gather_facts: yes

  tasks:
    - name: Test SSH connection
      ping:

    - name: Install httpd
      yum:
        name: httpd
        state: present
        update_cache: yes

    - name: Start and enable httpd service
      service:
        name: httpd
        state: started
        enabled: yes

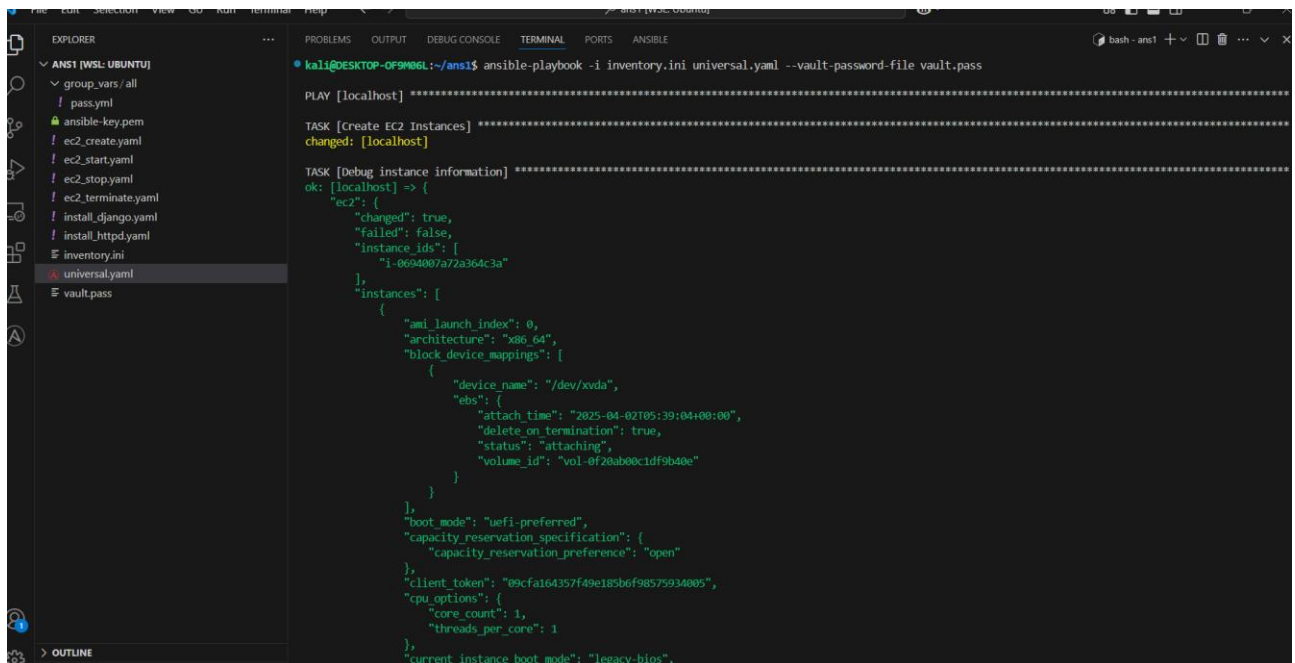
    - name: Install python3
      yum:
        name: python3
        state: present

    - name: Install pip
      yum:
        name: python3-pip
        state: present

    - name: Install Django
      pip:
        name: django
        state: present
        executable: pip3
```

Now execute the file ,

ansible-playbook -i inventory.ini ec2_terminate.yaml --vault-password-file vault.pass

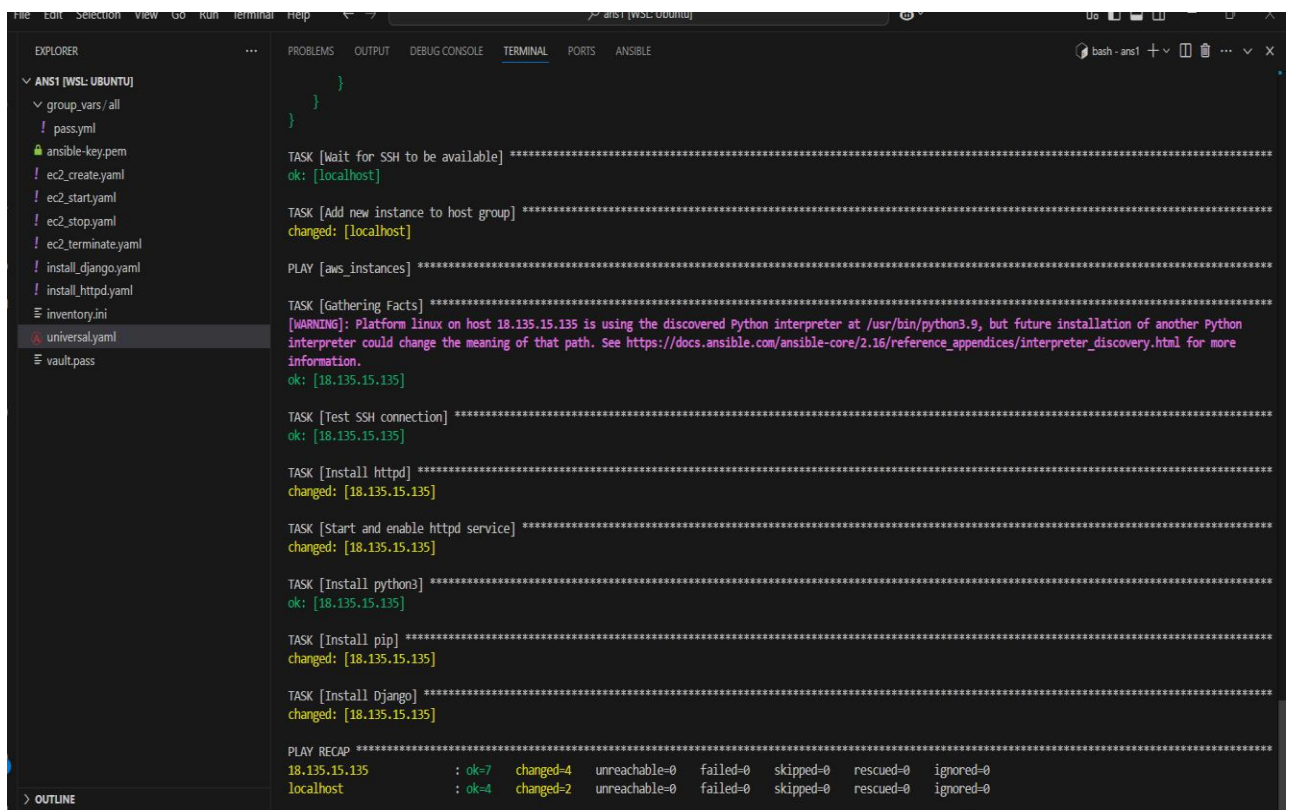


```
kali@DESKTOP-OF9W6GL:~/ans1$ ansible-playbook -i inventory.ini universal.yaml --vault-password-file vault.pass

PLAY [localhost] *****

TASK [Create EC2 Instances] *****
changed: [localhost]

TASK [Debug instance information] *****
ok: [localhost] => {
  "ec2": {
    "changed": true,
    "failed": false,
    "instance_ids": [
      "i-0694007a72a364c3a"
    ],
    "instances": [
      {
        "ami_launch_index": 0,
        "architecture": "x86_64",
        "block_device_mappings": [
          {
            "device_name": "/dev/xvda",
            "ebs": {
              "attach_time": "2025-04-02T05:39:04+00:00",
              "delete_on_termination": true,
              "status": "attaching",
              "volume_id": "vol-0f20ab00c1df9b40e"
            }
          }
        ],
        "boot_mode": "uefi-preferred",
        "capacity_reservation_specification": {
          "capacity_reservation_preference": "open"
        },
        "client_token": "09cfa164357f49e185b6f98575934005",
        "cpu_options": {
          "core_count": 1,
          "threads_per_core": 1
        },
        "current_instance_boot_mode": "legacy-bios",
```



```

    }
  }
}

TASK [Wait for SSH to be available] *****
ok: [localhost]

TASK [Add new instance to host group] *****
changed: [localhost]

PLAY [aws_instances] *****

TASK [Gathering Facts] *****
[WARNING]: Platform linux on host 18.135.15.135 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python
interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for more
information.
ok: [18.135.15.135]

TASK [Test SSH connection] *****
ok: [18.135.15.135]

TASK [Install httpd] *****
changed: [18.135.15.135]

TASK [Start and enable httpd service] *****
changed: [18.135.15.135]

TASK [Install python3] *****
ok: [18.135.15.135]

TASK [Install pip] *****
changed: [18.135.15.135]

TASK [Install Django] *****
changed: [18.135.15.135]

PLAY RECAP *****
18.135.15.135      : ok=7  changed=4  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
localhost         : ok=4  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```



```

---
- hosts: localhost
  vars_files:
    - group_vars/all/pass.yml
  connection: local
  gather_facts: no

  tasks:
    - name: Create 3 EC2 Instances
      amazon.aws.ec2_instance:
        name: "KGN-{{ item }}"
        key_name: "ansible-key"
        instance_type: "t2.micro"
        security_groups: ["default"]
        region: "eu-west-2"
        aws_access_key: "{{ ec2_access_key }}"
        aws_secret_key: "{{ ec2_secret_key }}"
        network:
          assign_public_ip: true
        image_id: "ami-0e56583ebfdcf098f"
        wait: yes
        wait_timeout: 600
      register: ec2
      loop: "{{ range(1, 4) }}" # Creates 3 instances (KGN-1, KGN-2, KGN-3)

    - name: Debug instance information
      debug:
        var: ec2

    - name: Add each new instance to host group
      add_host:
        hostname: "{{ item.instances[0].public_ip_address }}"
        groups: aws_instances
        ansible_user: ec2-user
        ansible_ssh_private_key_file: "~/ans1/ansible-key.pem"
      ansible_ssh_common_args: "-o StrictHostKeyChecking=no -o UserKnownHostsFile=/dev/null"
      loop: "{{ ec2.results }}" # Loop through all created instances

- hosts: aws_instances
  become: yes
  gather_facts: yes

  tasks:
    - name: Test SSH connection
      ping:

```

```
- name: Install httpd
  yum:
    name: httpd
    state: present
    update_cache: yes

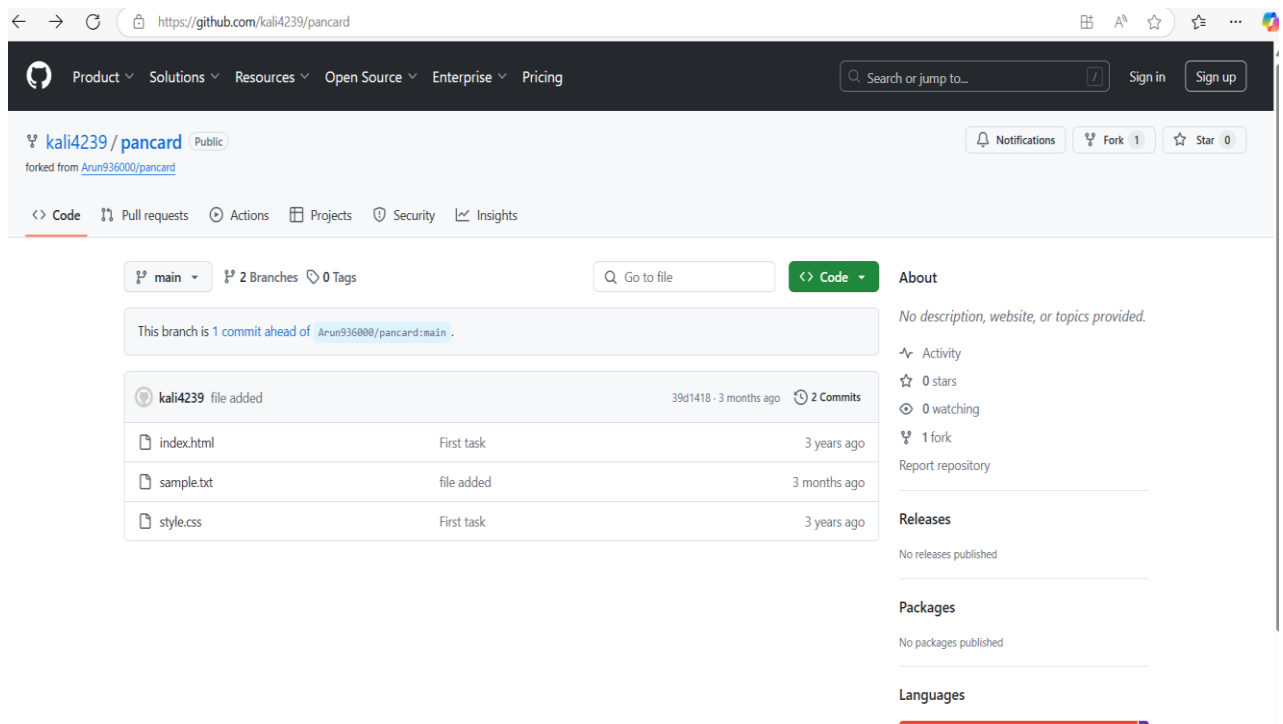
- name: Start and enable httpd service
  service:
    name: httpd
    state: started
    enabled: yes

- name: Install python3
  yum:
    name: python3
    state: present

- name: Install pip
  yum:
    name: python3-pip
    state: present

- name: Install Django
  pip:
    name: django
    state: present
    executable: pip3
```

Clone Github repository by using Playbook



This is my (pancard) repository . We going to be clone this repo .

Create yaml file to perform task here it was named as **github.yaml**

```
---
- hosts: localhost
  vars_files:
    - group_vars/all/pass.yml
  connection: local
  gather_facts: no

  tasks:
    - name: Create EC2 Instances
      amazon.aws.ec2_instance:
        name: "Github"
        key_name: "ansible-key"
        instance_type: "t2.micro"
        security_groups: ["default"]
        region: "eu-west-2"
        aws_access_key: "{{ ec2_access_key }}"
        aws_secret_key: "{{ ec2_secret_key }}"
        network:
          assign_public_ip: true
        image_id: "ami-0e56583ebfdcf098f"
        wait: yes
```

```

    wait_timeout: 600
    register: ec2

- name: Wait for SSH to be available
  wait_for:
    host: "{{ ec2.instances[0].public_ip_address }}"
    port: 22
    delay: 10
    timeout: 300
    connection: local

- name: Add new instance to host group
  add_host:
    hostname: "{{ ec2.instances[0].public_ip_address }}"
    groups: aws_instances
    ansible_user: ec2-user
    ansible_ssh_private_key_file: "~/ans1/ansible-key.pem"
    ansible_ssh_common_args: "-o StrictHostKeyChecking=no -o
UserKnownHostsFile=/dev/null"

- hosts: aws_instances
  become: yes
  gather_facts: yes

tasks:
  - name: Test SSH connection
    ping:

  - name: Install httpd
    yum:
      name: httpd
      state: present
      update_cache: yes

  - name: Start and enable httpd service
    service:
      name: httpd
      state: started
      enabled: yes

  - name: Install python3
    yum:
      name: python3
      state: present

  - name: Install pip

```

```

yum:
  name: python3-pip
  state: present

- name: Install Django
  pip:
    name: django
    state: present
    executable: pip3

- name: Install git
  yum:
    name: git
    state: present

- name: Verify git installation
  command: git --version
  register: git_check
  changed_when: false
  ignore_errors: yes

- name: Create directory for the application
  file:
    path: /var/www/myapp
    state: directory
    mode: '0755'

- name: Clone GitHub repository
  git:
    repo: "https://github.com/kali4239/pancard.git"
    dest: /var/www/myapp
    clone: yes
    update: no
    when: git_check.rc == 0

- name: Set proper permissions for the app directory
  file:
    path: /var/www/myapp
    owner: apache
    group: apache
    recurse: yes

```

As per our previous topics we can create insatance and install packages if needed then,

- Install git and check installation using `git --version`
- Then create directory & set path **`var/www/myapp`** instead of `myapp` we can give any name .
- **Mode : 0755**
 - **0** – is special bit , **7** – owner(apache) permission to read and write , **5** – group permission for read and write , **5** – is any one can read and write .
- **repo:** "**`https://github.com/kali4239/pancard.git`**" here give you repository like by providing username and repository name .
- **dest:** **`/var/www/myapp`** this is destination for store the cloning files.
- **when:** **`git_check.rc == 0`** it used to check return code 0 which means it return 0 git installed successfully , now clone files .

Now run the file ,

```
ansible-playbook -i inventory.ini ec2_github.yaml --vault-password-  
file vault.pass
```

The image shows a terminal window with a dark background. On the left, there is a file explorer sidebar showing the contents of a directory named 'ANS1 [WSL-UBUNTU]'. The files listed are: 'group_vars/all', 'pass.yml', 'ansible-key.pem', 'ec2_create.yml', 'ec2_start.yml', 'ec2_stop.yml', 'ec2_terminate.yml', 'github.yml' (highlighted), 'install_django.yml', 'install_httpd.yml', 'inventory.ini', 'universality.yml', and 'vault.pass'.

The main terminal area shows the output of an Ansible command: `ansible-playbook -i inventory.ini github.yml --vault-password-file vault.pass`. The output is color-coded and shows the following steps:

- PLAY [localhost]**: The first play runs on the localhost.
- TASK [Create EC2 Instances]**: This task is marked as 'changed' and runs on 'localhost'.
- TASK [Wait for SSH to be available]**: This task is marked as 'ok' and runs on 'localhost'.
- TASK [Add new instance to host group]**: This task is marked as 'changed' and runs on 'localhost'.
- PLAY [aws_instances]**: The second play runs on the 'aws_instances' group.
- TASK [Gathering Facts]**: This task runs on the remote host. A warning message is displayed: `[WARNING]: Platform linux on host 18.171.147.11 is using the discovered Python interpreter at /usr/bin/python3.9, but future installation of another Python interpreter could change the meaning of that path. See https://docs.ansible.com/ansible-core/2.16/reference_appendices/interpreter_discovery.html for more information.` The task is marked as 'ok'.
- TASK [Test SSH connection]**: This task is marked as 'ok'.
- TASK [Install httpd]**: This task is marked as 'changed'.
- TASK [Start and enable httpd service]**: This task is marked as 'changed'.
- TASK [Install python3]**: This task is marked as 'ok'.
- TASK [Install pip]**: This task is marked as 'changed'.
- TASK [Install Django]**: This task is marked as 'changed'.
- TASK [Install git]**: The final task shown, marked as 'ok'.

At the bottom of the terminal, there are tabs for 'OUTLINE' and 'TIMELINE'.

```
Information.
ok: [18.171.147.11]

TASK [Test SSH connection] *****
ok: [18.171.147.11]

TASK [Install httpd] *****
changed: [18.171.147.11]

TASK [Start and enable httpd service] *****
changed: [18.171.147.11]

TASK [Install python3] *****
ok: [18.171.147.11]

TASK [Install pip] *****
changed: [18.171.147.11]

TASK [Install Django] *****
changed: [18.171.147.11]

TASK [Install git] *****
changed: [18.171.147.11]

TASK [Verify git installation] *****
ok: [18.171.147.11]

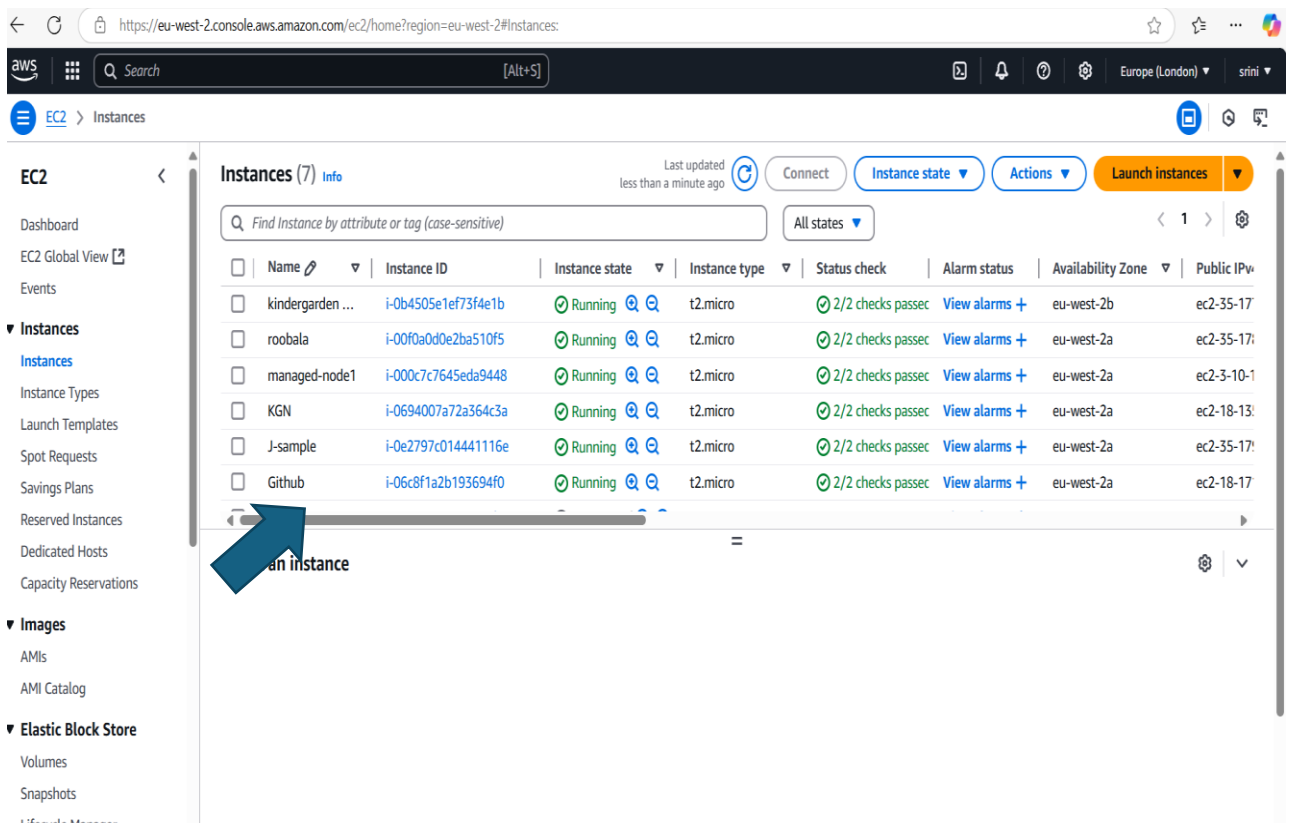
TASK [Create directory for the application] *****
changed: [18.171.147.11]

TASK [Clone Github repository] *****
changed: [18.171.147.11]

TASK [Set proper permissions for the app directory] *****
changed: [18.171.147.11]

PLAY RECAP *****
18.171.147.11 : ok=12 changed=8 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
localhost : ok=3 changed=2 unreachable=0 failed=0 skipped=0 rescued=0 ignored=0
```

Now all tasks are complete successfully . Verify in aws console also ,



After that login the instance via wsl or bitwise


```

    security_groups: ["default"]
    region: "eu-west-2"
    aws_access_key: "{{ ec2_access_key }}"
    aws_secret_key: "{{ ec2_secret_key }}"
    network:
        assign_public_ip: true
    image_id: "ami-0e56583ebfdcf098f"
    wait: yes
    wait_timeout: 600
    register: ec2

- name: Wait for SSH to be available
  wait_for:
    host: "{{ ec2.instances[0].public_ip_address }}"
    port: 22
    delay: 10
    timeout: 300
    connection: local

- name: Add new instance to host group
  add_host:
    hostname: "{{ ec2.instances[0].public_ip_address }}"
    groups: aws_instances
    ansible_user: ec2-user
    ansible_ssh_private_key_file: "~/ans1/ansible-key.pem"
    ansible_ssh_common_args: "-o StrictHostKeyChecking=no -o
UserKnownHostsFile=/dev/null"

- hosts: aws_instances
  become: yes
  gather_facts: yes

tasks:
  - name: Test SSH connection
    ping:

  - name: Install httpd
    yum:
      name: httpd
      state: present
      update_cache: yes

  - name: Start and enable httpd service
    service:
      name: httpd
      state: started

```

```
enabled: yes

- name: Install python3
  yum:
    name: python3
    state: present

- name: Install pip
  yum:
    name: python3-pip
    state: present

- name: Install Django
  pip:
    name: django
    state: present
    executable: pip3

- name: Install git
  yum:
    name: git
    state: present

- name: Verify git installation
  command: git --version
  register: git_check
  changed_when: false
  ignore_errors: yes

- name: Create directory for the application
  file:
    path: /var/www/myapp
    state: directory
    mode: '0755'

- name: Clone GitHub repository
  git:
    repo: "https://github.com/kali4239/pancard.git"
    dest: /var/www/myapp
    clone: yes
    update: no
  when: git_check.rc == 0

- name: Check if index.html exists in repository
  stat:
    path: /var/www/myapp/index.html
```

```
register: index_file

- name: Move index.html to web root
  copy:
    src: /var/www/myapp/index.html
    dest: /var/www/html/index.html
    remote_src: yes
    owner: apache
    group: apache
    mode: '0644'
  when: index_file.stat.exists

- name: Set proper permissions for the app directory
  file:
    path: /var/www/myapp
    owner: apache
    group: apache
    recurse: yes
```

Now run the file ,

ansible-playbook -i inventory.ini github.yaml --vault-password-file vault.pass

Clone django project & runserver via Playbook

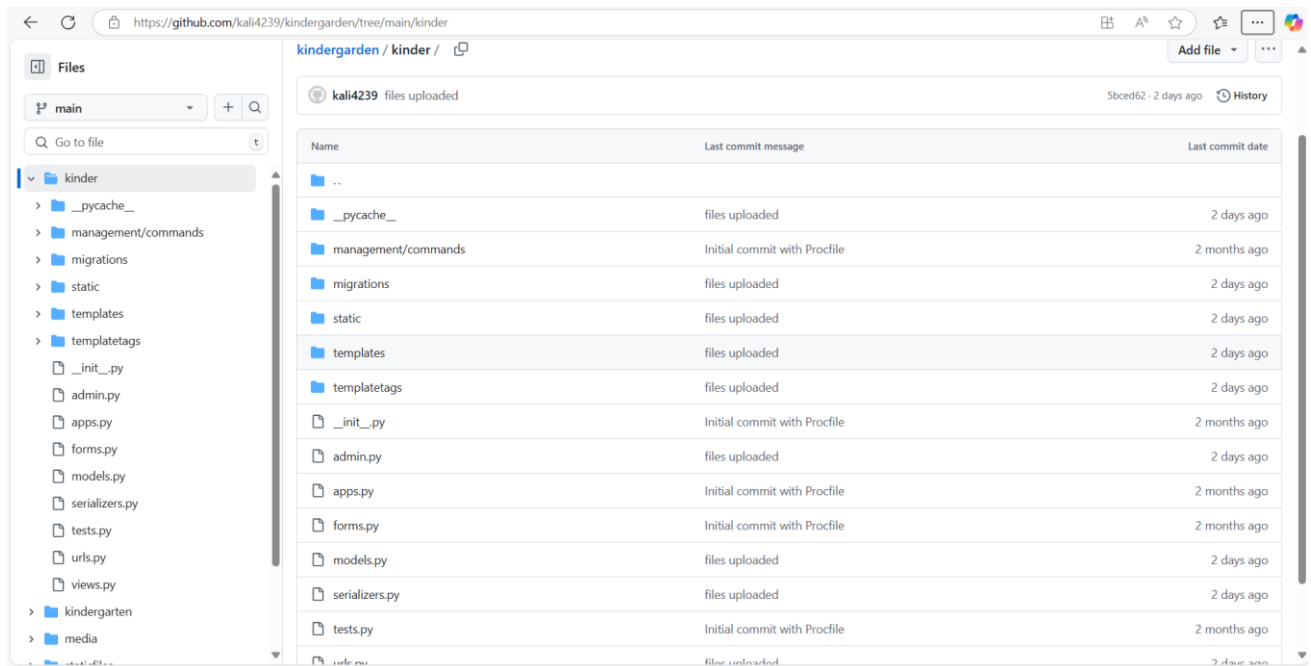
This is my github repository here i have kindergarden website

The screenshot shows a GitHub repository named 'kindergarden' by user 'kali4239'. The repository is public and has 1 branch (main) and 0 tags. The file list shows the following files and their commit history:

File	Commit Message	Time
kinder	files uploaded	47 minutes ago
kindergarten	files uploaded	47 minutes ago
media	files uploaded	47 minutes ago
staticfiles	Initial commit with Procfiler	2 months ago
db.sqlite3	files uploaded	47 minutes ago
manage.py	Initial commit with Procfiler	2 months ago
requirements.txt	Initial commit	2 months ago

The repository also has a README section with a placeholder for a book icon and the text 'Add a README'. On the right side, there are sections for 'About' (No description, website, or topics provided), 'Activity' (0 stars, 1 watching, 0 forks), 'Releases' (No releases published, Create a new release), 'Packages' (No packages published, Publish your first package), and 'Languages' (HTML 40.5%, CSS 21.3%, Python 20.5%, JavaScript 17.7%).

Here **kindergarten** is project name , **kinder** is app name .



This is inside the app .

Create new yaml file for perform task . Here name as **djproject.yaml**

```
- hosts: localhost
  vars_files:
    - group_vars/all/pass.yml
  connection: local
  gather_facts: no

  tasks:
    - name: Create EC2 Instances
      amazon.aws.ec2_instance:
        name: "KN"
        key_name: "ansible-key"
        instance_type: "t2.micro"
        security_groups: ["default"]
        region: "eu-west-2"
        aws_access_key: "{{ ec2_access_key }}"
        aws_secret_key: "{{ ec2_secret_key }}"
        network:
          assign_public_ip: true
        image_id: "ami-0e56583ebfdcf098f"
        wait: yes
        wait_timeout: 600
      register: ec2

    - name: Wait for SSH to be available
      wait_for:
        host: "{{ ec2.instances[0].public_ip_address }}"
```

```

    port: 22
    delay: 10
    timeout: 300
    connection: local

- name: Add new instance to host group
  add_host:
    hostname: "{{ ec2.instances[0].public_ip_address }}"
    groups: aws_instances
    ansible_user: ec2-user
    ansible_ssh_private_key_file: "~/ans1/ansible-key.pem"
    ansible_ssh_common_args: "-o StrictHostKeyChecking=no -o
UserKnownHostsFile=/dev/null"

- hosts: aws_instances
  become: yes
  gather_facts: yes

tasks:
  - name: Test SSH connection
    ping:

  - name: Install required system packages
    yum:
      name:
        - httpd
        - python3
        - python3-pip
        - git
      state: present
      update_cache: yes

  - name: Clone GitHub repository
    git:
      repo: "https://github.com/kali4239/kindergarden.git"
      dest: /opt/kindergarden
      clone: yes
      update: no

  - name: Install all required Python packages
    pip:
      name:
        - django==4.2.10
        - djangorestframework
        - Pillow
        - psycopg2-binary

```

```

- django-widget-tweaks
  executable: pip3

- name: Install requirements from requirements.txt
  pip:
    requirements: /opt/kindergarden/requirements.txt
    executable: pip3
    ignore_errors: yes

- name: Run migrations
  command: python3 manage.py migrate
  args:
    chdir: /opt/kindergarden
  ignore_errors: yes

- name: Collect static files
  command: python3 manage.py collectstatic --noinput
  args:
    chdir: /opt/kindergarden
  ignore_errors: yes

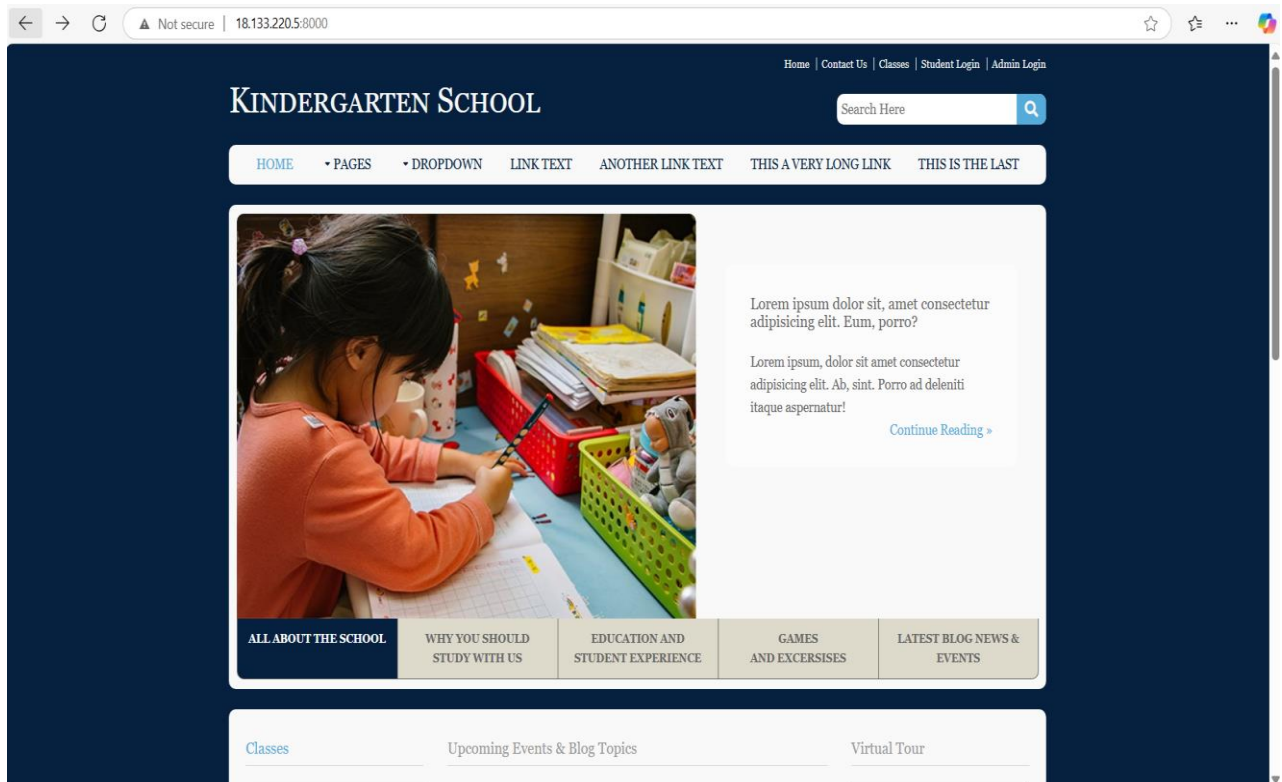
- name: Run Django development server
  command: python3 manage.py runserver 0.0.0.0:8000
  async: 7200
  poll: 0
  args:
    chdir: /opt/kindergarden

```

- **dest: /opt/kindergarden** for copy the cloned repo in opt (optional) - because by default third – party app is stored opt .
- **- psycopg2-binary** this is database adapter .
- **- django-widget-tweaks** it was used in our project so i had install this for link forms .
- **async: 7200 , poll: 0** it used to continously runserver 2hr without waiting .
- **chdir: /opt/kindergarden** for go to project folder before running .

Now run the playbook ,

ansible-playbook -i inventory.ini djproject.yaml --vault-password-file vault.pass



So now we can see project output on public IP .

Ansible GUI

It is a Graphical User Interface . Web bases ui for execute the ansible operations. Visualize, manage,execute playbook instead of (CLI) command line interface which means terminal based workflows .

Why GUI ?

Visualization – you can see the result in neat dashboard , **user-friendly** , **centralized control** – manage multiple projects in one place .

Some of GUI tools are ,

Red Hat Ansible Automation Platform, Awx – Free & open Source version of Ansible Tower. So now we are going to be use **AWX** Ansible worx (web based gui)

Installation

Open wsl ,

Sudo apt update && Sudo apt upgrade -y for update all existing packages .

Install Docker in wsl

Sudo apt install -y docker.io

Sudo systemctl enable docker

Sudo systemctl start docker

Add user ,

Sudo usermod -aG docker \$user

newgrp docker

Install docker compose ,

Sudo apt install -y docker-compose

Install python required packages,

Sudo apt install -y python3-pip git

Pip3 install docker-compose

Install Node.js (optional)

Curl -fsSL https://deb.nodesource.com/setup_1ts.x|sudo -E base -

Sudo apt install -y node js

Clone AWX repository ,

git clone https://github.com/ansible/awx.git

cd awx

git checkout

Install Awx,

cd installer

nano inventory (then uncomment the `admin_password` , also note the password for later access dashboard)

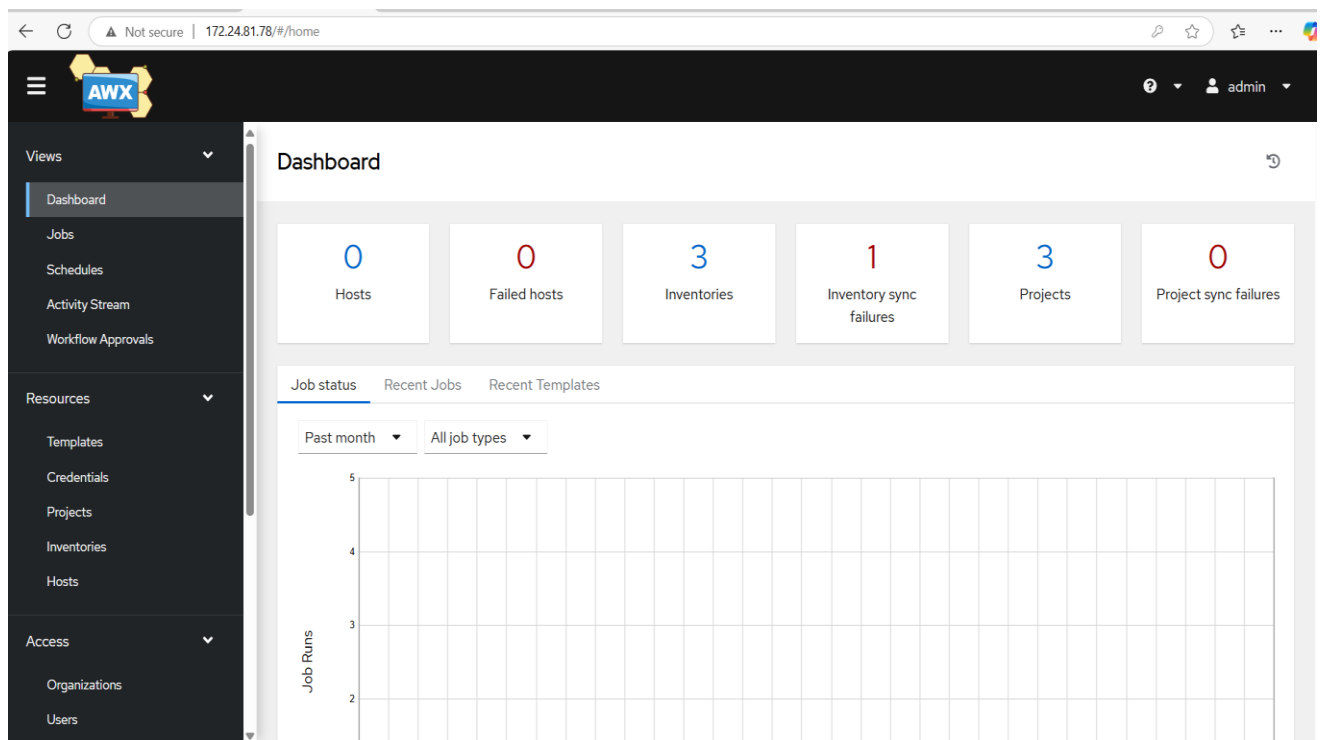
Check IP ,

hostname -I

It will return ip address copy & paste it on your browser . Now you can see the awx dashboard on browser .

Username : admin

Password : (As you given during creation)



Note :

If suppose the login portal show invalid username use the following command to create superuser .

docker exec -it awx_web bash

awx-manage createsuperuser

Then give the username and password .